Local AI Coding Setup — Cheat Sheet

Everything we configured: Ollama + NVIDIA GPU + Continue.dev in VS Code.

Table of Contents

1. Docker Compose Setup (GPU)
2. Alternative: Native Install (curl | sh)
3. Alternative: User-Local Install (No Sudo)
4. Alternative: Docker Run (One-Liner)
5. Continue.dev Config
6. Ollama Model Commands
7. Debug & Verify Commands
8. NVIDIA GPU Troubleshooting
9. VS Code Shortcuts
10. Useful Links
11. Model Size Reference
12. Hybrid Setup

---

1. Docker Compose Setup (GPU)

Best for: Reproducible, isolated, easy to remove.

`docker-compose.yml`

```
services:
  ollama:
    image: ollama/ollama
    container_name: ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    restart: unless-stopped

volumes:
  ollama:
```

Start / Stop

```
docker compose up -d          # Start
```

```
docker compose down           # Stop
docker compose down -v        # Stop + delete models (careful!)
```

---

2. Alternative: Native Install (curl | sh)

Best for: Direct system integration, systemd service, fastest startup.

> ⚠️ Review the script first: `curl -fsSL https://ollama.com/install.sh > install.sh && less install.sh`

Install

```
curl -fsSL https://ollama.com/install.sh | sh
```

Start / Stop Service

```
sudo systemctl start ollama
sudo systemctl stop ollama
sudo systemctl enable ollama   # Auto-start on boot
```

Verify

```
ollama --version
ollama list
ollama serve                   # Run manually (foreground)
```

Uninstall

```
sudo rm /usr/local/bin/ollama
sudo rm -rf /usr/share/ollama
sudo userdel ollama
sudo groupdel ollama
sudo rm /etc/systemd/system/ollama.service
```

---

3. Alternative: User-Local Install (No Sudo)

Best for: Shared machines, no root access, maximum isolation.

```
# Download binary to home directory
curl -L https://ollama.com/download/Ollama-linux-amd64.tgz -o ollama.tgz
tar -xzf ollama.tgz -C ~/.local/

# Add to PATH
export PATH="$HOME/.local/bin:$PATH"
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc

# Run manually (no systemd)
ollama serve
```

Models stored at: `~/.ollama/models`

---

4. Alternative: Docker Run (One-Liner)

Best for: Quick testing without a compose file.

CPU Only

```
docker run -d   -v ollama:/root/.ollama   -p 11434:11434   --name ollama   ollama/ollama
```

With NVIDIA GPU

```
docker run -d   --gpus all   -v ollama:/root/.ollama   -p 11434:11434   --name ollama   ollama/ollama
```

With Specific GPU (multi-GPU systems)

```
docker run -d   --gpus '"device=0"'   -v ollama:/root/.ollama   -p 11434:11434   --name ollama   ollama/
↪  ollama
```

Docker Image Variants

Image	Use Case
`ollama/ollama`	Default (CUDA for NVIDIA)
`ollama/ollama:rocm`	AMD GPUs (ROCm)
`ollama/ollama:cpu`	CPU-only (smaller image)

---

5. Continue.dev Config (`~/.continue/config.yaml`)

```
name: Local Ollama
version: 1.0.0
schema: v1

models:
  - name: Qwen Coder 7B
    provider: ollama
    model: qwen2.5-coder:7b
    apiBase: http://localhost:11434
    roles:
      - chat
      - edit
      - apply
      - agent

  - name: Qwen Coder 3B
    provider: ollama
    model: qwen2.5-coder:3b
```

```
    apiBase: http://localhost:11434
    roles:
      - autocomplete

context:
  - provider: codebase
  - provider: workspace
```

Location: `~/.continue/config.yaml`
Reload after changes: `Ctrl+Shift+P` → `Developer: Reload Window`

---

6. Ollama Model Commands

Native Install Commands

Command	Action
`ollama pull qwen2.5-coder:7b`	Download 7B model
`ollama pull qwen2.5-coder:3b`	Download 3B model
`ollama pull qwen2.5-coder:14b`	Download 14B model (needs 16GB VRAM)
`ollama list`	List installed models
`ollama rm <model>`	Remove a model
`ollama run qwen2.5-coder:7b`	Interactive chat
`ollama ps`	Show running models
`ollama stop qwen2.5-coder:7b`	Stop a running model

Docker Container Commands

Command	Action
`docker exec -it ollama ollama pull qwen2.5-coder:7b`	Download 7B model into container
`docker exec -it ollama ollama pull qwen2.5-coder:3b`	Download 3B model into container
`docker exec -it ollama ollama pull qwen2.5-coder:14b`	Download 14B model into container
`docker exec ollama ollama list`	List models inside container
`docker exec ollama ollama rm <model>`	Remove model from container
`docker exec -it ollama ollama run qwen2.5-coder:7b`	Interactive chat in container
`docker exec ollama ollama ps`	Show running models in container

---

7. Debug & Verify Commands

Check Ollama is running

```
curl http://localhost:11434/api/tags
```

Check GPU is used (native)

```
nvidia-smi
```

Check GPU is used (Docker)

```
docker exec ollama nvidia-smi
```

Test model generation

```
curl http://localhost:11434/api/generate -d '{
  "model": "qwen2.5-coder:7b",
  "prompt": "Write a Python function to reverse a string"
}'
```

Check Continue config

```
cat ~/.continue/config.yaml
```

Check for conflicting project configs

```
find . -name "config.yaml" -path "*/.continue/*" 2>/dev/null
```

---

8. NVIDIA GPU Troubleshooting

8.1 Quick Diagnostic

Run this all-in-one check:

```
echo "=== Host GPU ===" && nvidia-smi && echo -e "
=== Driver Version ===" && cat /proc/driver/nvidia/version && echo -e "
=== Docker Version ===" && docker --version && echo -e "
=== NVIDIA Container Toolkit ===" && dpkg -l | grep nvidia-container && echo -e "
=== Docker Daemon Config ===" && sudo cat /etc/docker/daemon.json && echo -e "
=== GPU Inside Container ===" && docker run --rm --gpus all nvidia/cuda:12.0-base nvidia-smi
```

8.2 Error: `could not select device driver "nvidia"`

Solution for Ubuntu 20.04:

```
# Step 1: Install NVIDIA Container Toolkit
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit

# Step 2: Configure Docker daemon
sudo tee /etc/docker/daemon.json <<'EOF'
{
    "runtimes": {
        "nvidia": {
            "path": "nvidia-container-runtime",
```

```
            "runtimeArgs": []
        }
    }
}
EOF

# Step 3: Restart Docker
sudo systemctl restart docker

# Step 4: Verify
docker run --rm --gpus all nvidia/cuda:12.0-base nvidia-smi
```

8.3 Alternative: NVIDIA as Default Runtime

If `--gpus all` still fails, make NVIDIA the default:

```
sudo tee /etc/docker/daemon.json <<'EOF'
{
    "default-runtime": "nvidia",
    "runtimes": {
        "nvidia": {
            "path": "nvidia-container-runtime",
            "runtimeArgs": []
        }
    }
}
EOF
sudo systemctl restart docker
```

8.4 Verify NVIDIA Container Toolkit Installation

```
# Check toolkit packages
dpkg -l | grep -E "nvidia-container-toolkit|nvidia-docker2"

# Check runtime binary exists
which nvidia-container-runtime
nvidia-container-runtime --version

# Check Docker can see the runtime
docker info | grep -i nvidia
```

8.5 Check CUDA Compatibility

```
# Host CUDA version
nvcc --version

# Driver-supported CUDA version
nvidia-smi | grep "CUDA Version"

# Container CUDA version
```

```
docker run --rm --gpus all nvidia/cuda:12.0-base bash -c "nvcc --version"
```

8.6 GPU Not Showing Inside Container

```
# Check if nvidia modules are loaded
lsmod | grep nvidia

# Check device permissions
ls -la /dev/nvidia*

# Add user to video group (if permission denied)
sudo usermod -aG video $USER
# Log out and back in
```

8.7 Multi-GPU: Use Specific GPU

```
# List GPUs
nvidia-smi -L

# Docker: use GPU 0 only
docker run -d --gpus '"device=0"' ...

# Docker: use GPUs 0 and 1
docker run -d --gpus '"device=0,1"' ...

# Docker Compose: specific GPU
# In docker-compose.yml:
# deploy:
#   resources:
#     reservations:
#       devices:
#         - driver: nvidia
#           device_ids: ['0']
#           capabilities: [gpu]
```

8.8 Ollama Not Using GPU (Falls Back to CPU)

```
# Check if Ollama detects CUDA
docker logs ollama | grep -i cuda
docker logs ollama | grep -i gpu

# Force GPU by checking environment
docker exec ollama env | grep -i cuda

# Restart with explicit GPU
docker compose down
docker compose up -d
```

8.9 Full Reset (Nuclear Option)

```
# Stop everything
docker compose down

# Purge Docker cache
docker system prune -a

# Reinstall NVIDIA Container Toolkit
sudo apt-get remove --purge nvidia-container-toolkit
sudo apt-get install nvidia-container-toolkit

# Reset Docker config
sudo rm /etc/docker/daemon.json
sudo systemctl restart docker

# Rebuild from scratch
docker compose up -d
```

---

9. VS Code Shortcuts (Continue.dev)

Shortcut	Action
`Ctrl + L`	Open Continue chat panel
`Ctrl + I`	Inline edit
`Tab`	Accept autocomplete
`@workspace`	Inject project context
`@codebase`	Inject entire codebase context
`Ctrl+Shift+P` → `Continue: Reload Window`	Reload Continue config

---

10. Useful Links

Resource	URL
Ollama Docker Hub	https://hub.docker.com/r/ollama/ollama
Ollama Models Library	https://ollama.com/library
Ollama GitHub	https://github.com/ollama/ollama
Continue.dev Docs	https://docs.continue.dev
Continue.dev Config Reference	https://docs.continue.dev/reference
NVIDIA Container Toolkit Install	https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html
Kimi API Platform	https://platform.kimi.ai
Qwen2.5-Coder on HuggingFace	https://huggingface.co/Qwen/Qwen2.5-Coder-7B

---

11. Quick Model Size Reference

Model	VRAM Needed	Speed	Best For
`qwen2.5-coder:1.5b`	4 GB	Very fast	Laptops, quick suggestions
`qwen2.5-coder:3b`	6 GB	Fast	Autocomplete, simple chat
`qwen2.5-coder:7b`	8 GB	Fast	Daily coding, chat, agent (basic)
`qwen2.5-coder:14b`	16 GB	Moderate	Better agent mode, complex tasks
`qwen2.5-coder:32b`	24 GB+	Slower	Maximum quality, local only

---

12. Hybrid Setup (Local + Cloud)

Use local for speed/privacy, cloud for heavy agent tasks:

```
models:
  - name: Qwen Coder 7B
    provider: ollama
    model: qwen2.5-coder:7b
    apiBase: http://localhost:11434
    roles:
      - chat
      - edit
      - apply
      - autocomplete

  - name: Kimi K2.6
    provider: openai
    model: kimi-k2-6
    apiBase: https://api.moonshot.cn/v1
    apiKey: ${KIMI_API_KEY}
    roles:
      - agent
```

Set API key in your shell:

```
export KIMI_API_KEY=your_key_here
echo 'export KIMI_API_KEY=your_key_here' >> ~/.bashrc
```

Generated on 2026-08-02

Generated by Kimi.ai